

Sportiva CM Explained Like You Are a Beginner

1. The big idea in one sentence

Sportiva CM is a Django web application that helps sports clubs, athletes, fans, sponsors, and organizers work together in one online place.

Think of it like this:

- a place where clubs create their profile,
- athletes can register,
- events can be announced,
- fans can follow updates,
- products can be sold,
- sponsors can support campaigns,
- and everything is stored in a database.

2. What is Django?

Django is a Python framework used to build websites quickly.

It gives you:

- a project structure,
- database tools,
- login and user management,
- URL routing,
- templates for pages,
- and admin support.

In very simple words:

Django is the engine that powers the whole system.

3. The project starts here: manage.py

Reference: ../manage.py

This file is the starter key of the project.

When you run:

```
python manage.py runserver
```

Django starts the web application.

Here is the flow:

You type command

|

v

manage.py

|

v

Django loads project settings

|

v

Your app starts in the browser

The important part is this:

- it tells Django where the project settings file is,
- then Django loads the whole app,
- then the app runs on a local server.

4. Project settings: sportiva_cm/settings.py

Reference: ../sportiva_cm/settings.py

This file is the brain of the project.

It tells Django:

- the project name,
- which apps are installed,
- the database to use,
- where static files are,
- where media files are stored,
- and which table is used for users.

Important line:

```
AUTH_USER_MODEL = 'accounts.User'
```

This means:

- the project does not use the default Django user,
- it uses the custom user model from the accounts app,
- so the system can have different roles like athlete, sponsor, trainer, and organization.

Simple diagram:

settings.py

|

+--> INSTALLED_APPS

+--> DATABASES

+--> AUTH_USER_MODEL

+--> STATIC_URL / MEDIA_URL

+--> LOGIN_URL / REDIRECTS

5. URLs: sportiva_cm/urls.py

Reference: ../sportiva_cm/urls.py

This file is like the map of the whole website.

It decides which page opens when the user visits a path.

Example:

http://127.0.0.1:8000/admin/ --> Django admin panel

http://127.0.0.1:8000/ --> home page

http://127.0.0.1:8000/accounts/ --> accounts app routes

http://127.0.0.1:8000/events/ --> event routes

http://127.0.0.1:8000/marketplace/ --> marketplace routes

This is how Django matches a URL to the right page.

6. The accounts app: the user system

Reference: ../accounts/models.py

Reference: ../accounts/views.py

Reference: ../accounts/urls.py

This is the place where user accounts are created.

6.1 The custom user model

The project uses a custom model called User.

This allows different user roles:

- Athlete
- Organization
- Trainer
- Sponsor

The model stores:

- username,
- email,
- role,
- city,
- phone number,
- bio,
- avatar,
- verification status.

So instead of only one kind of user, we can have many types in one system.

6.2 Registration flow

User opens register page

|

v

Fills form

|

v

View receives data

|

v

Checks password, email, username

|

v

Creates user in database

|

v

User logs in automatically

|

v

Redirect to home page

The register_view function does the checking.

It verifies:

- username is not empty,
- email is present,
- password matches confirmation,
- password is long enough,
- username/email are unique,
- then a user is created.

6.3 Login flow

User opens login page

|

v

Enters username/password

|

v

authenticate() checks database

|

v

If valid: login(request, user)

|

v

User enters the app

The login system is simple and safe because Django handles the authentication.

6.4 Profile page

The profile page is protected with @login_required.

That means you cannot visit the profile if you are not logged in.

It shows:

- total events attended,
- number of posts,
- whether the user has an organization,
- and recent posts.

7. Home page: core app

Reference: ../core/views.py

Reference: ../templates/core/home.html

The core app is like the front door of the platform.

When a user visits the home page, the home view renders the page and sends data like:

- number of athletes,
- number of clubs,
- number of events,
- money sponsorship raised.

This page is the landing page and gives the impression of a big sports community website.

8. Organizations: building club profiles

Reference: ../organizations/models.py

This app is about clubs, academies, and sports organizations.

The central model is OrganizationProfile.

It contains:

- organization name,
- category (football, basketball, athletics, etc.),

- city,
- address,
- latitude and longitude,
- description,
- phone,
- WhatsApp,
- email,
- verification status.

This is important because organizations need to appear online like real businesses.

Example:

Organization Profile

|

+--> Name: Canon Yaoundé Youth Academy

+--> Sport: Football

+--> City: Yaoundé

+--> WhatsApp: +237699000001

+--> Verified: True

This helps fans find clubs and sponsors find teams.

9. Events: sports activities

Reference: ../events/models.py

The events app handles competitions, tournaments, and sports gatherings.

Important models:

- EventCategory
- Event
- EventAttendance

Event model contains:

- title,
- organizer,
- organization,
- category,
- sport,
- location,
- venue,
- start date,
- end date,
- description,
- banner image,
- WhatsApp contact,
- entry fee,
- max participants.

Event attendances

The app records who joins an event.

This is done with EventAttendance.

User A attends Event 5

|

+--> registration is saved in database

+--> user can see attendance record

+--> event organizer can count participants

So the system is not just for posting information; it also records action.

10. Media feed: social sports community

Reference: ../media_feed/models.py

This app works like a social media wall for the sports community.

Important models:

- Post
- Comment
- Like

A post can be:

- text,
- image,
- video,
- article.

A user can:

- publish a post,
- comment on a post,
- like a post,
- add hashtags.

Example:

Post by user: "Training session in Yaoundé"

|

+--> content

+--> image

+--> hashtags #Football #Training

+--> likes

+--> comments

This creates life and engagement in the platform.

11. Marketplace: selling sports gear

Reference: ../marketplace/models.py

The marketplace is for sports products such as equipment, boots, kits, balls, and training gear.

Important model: Product

It stores:

- seller,
- category,
- title,
- description,
- price,
- city,
- condition,
- image,
- WhatsApp number,
- availability.

This helps local sports communities buy and sell products easily.

Example:

Product: Football boots

|

+--> seller: athlete or club

+--> price: 20,000 FCFA

+--> city: Douala

+--> condition: Good

+--> WhatsApp contact available

This creates economic activity inside the sports ecosystem.

12. Sponsorships: funding sports projects

Reference: ../sponsorships/models.py

This app allows clubs or organizations to ask for support.

Important models:

- Campaign
- Pledge

Campaign example

Campaign Title: "Support the U-17 Football Team"

|

---> category: Travel & Transport

---> target amount: 5,000,000 FCFA

---> raised amount: 2,300,000 FCFA

---> city: Yaoundé

A sponsor can make a contribution.

The system saves the pledge and shows the progress.

This helps sports projects get practical support.

13. The full project architecture

Browser

|

v

Django Web Server

|

v

URL Router

|
+--> accounts app
+--> organizations app
+--> events app
+--> media_feed app
+--> marketplace app
+--> sponsorships app
+--> core app

|
v
Database (SQLite)

|
+--> Users
+--> Organizations
+--> Events
+--> Posts
+--> Products
+--> Campaigns

This is the basic system flow.

14. How data flows in the project

Example: user registers and creates an event.

User enters registration form

|
v
accounts/views.py

|
v
User model saves account

|
v
User logs in
|
v
User creates event in events app
|
v
Event is saved to database
|
v
Other users can see it on the website

This is the whole cycle:

- user enters data,
- view receives data,
- model saves data,
- database stores data,
- templates display the result.

15. What Django is actually doing behind the scenes

Every request goes through this flow:

User clicks link or submits form

|
v
Browser sends HTTP request

|
v
URL pattern matches a route

|

v

Django calls the correct view

|

v

View reads or saves data from models

|

v

Template renders HTML page

|

v

Browser shows the final result

This is the heart of the web application.

16. Relationship between files in simple language

manage.py

|

+--> settings.py

+--> urls.py

+--> apps

+--> models.py

+--> views.py

+--> urls.py

+--> templates/

That is the structure of a Django project.

17. Why this project matters in real life

This project matters because many sports actors need digital visibility.

Problems solved:

- clubs are invisible online,
- events are not well promoted,
- sponsorships are hard to organize,
- athletes do not have a place to showcase themselves,
- fans do not have a central community,
- sports products are not easy to trade.

Sportiva CM fixes these problems by consolidating everything into one platform.

18. The system as a real-world ecosystem

Athletes Clubs Sponsors Fans

||||

+----- connect through Sportiva CM -----+

|

v

Events, posts, products, campaigns

|

v

Cameroonian sports ecosystem

This means the website is not only code; it is a digital sports community.

19. Full running process from start to finish

1. Install Python and Django
2. Open the project folder
3. Activate virtual environment
4. Run: `python manage.py migrate`

5. Run: `python manage.py runserver`
6. Open browser
7. Register account
8. Login
9. Visit events, organizations, marketplace, sponsorships
10. Use the platform
11. Data is stored in SQLite
12. Admin can manage the application

This is the real working life of the project.

20. Final conclusion

The code is not magic.

It is a combination of:

- project setup,
- app creation,
- models,
- views,
- URLs,
- templates,
- database,
- user logic,
- and user interaction.

Every part is important.

If we remove one part, the whole system breaks.

So when you look at this project, think of it like a big house:

- `manage.py` is the key,

- settings.py is the blueprint,
- urls.py is the map,
- models.py is the structure,
- views.py is the action,
- templates are the rooms,
- and the database stores everything inside.

That is Sportiva CM.

21. Reference files

- ../manage.py
- ../sportiva_cm/settings.py
- ../sportiva_cm/urls.py
- ../accounts/models.py
- ../accounts/views.py
- ../accounts/urls.py
- ../core/views.py
- ../organizations/models.py
- ../events/models.py
- ../media_feed/models.py
- ../marketplace/models.py
- ../sponsorships/models.py

22. Very short beginner summary

Sportiva CM is like a sports social network + club directory + event manager + marketplace + sponsorship platform.

It is built with Django, Python, SQLite, HTML, CSS, and JavaScript.

The project takes user data, stores it in a database, and shows it on web pages.

Everything is connected.

That is the whole idea.

Prepared for learning and defense support.